FUTURE PROOF SOFTWARE SYSTEMS

EXERCISE 1 - G15

PET CLINIC SPRING BOOT APPLICATION - TASK 1

Task Overview

Objective:

Enhance the Spring PetClinic system by implementing new features and maintaining the software.

SET-1:

a) Show calculated prices + Owner names in Contract overview

b) Clicking on a Contract shows the list of its UsedItems

c) Enable adding new Contracts to Owners

d) Enable adding Items from the Storage to Contracts. They become UsedItems

SET-2:

a) Architecture of the System
b) Design of the System
c) C4 Level 1: System Context
d) C4 Level 2: Container
e) C4 Level 3: Component
f) C4 Level 4: Dynamic Flow
g) Code Quality
h) Testing

Difficulties Faced

1. **GitHub clone via terminal** — IntelliJ GUI authentication failed for cloning the private repository. Had to clone manually via terminal using a Personal Access Token (PAT). All Git operations (clone, commit, push, branch) had to be done via terminal.
2. **Gradle vs Maven confusion** — IntelliJ kept picking up Gradle instead of Maven. Had to manually add as Maven project.
3. **Spring Java Format violations** — had to run `./mvnw spring-javaformat:apply` before build because teammates' code had formatting issues.
4. **49 pom.xml warnings** — missing plugins, encoding errors in pom.xml

Tasks Implemented

Tasks Implemented:

a) **Display Contract Overview -** Show contract details including owner name and total calculated price

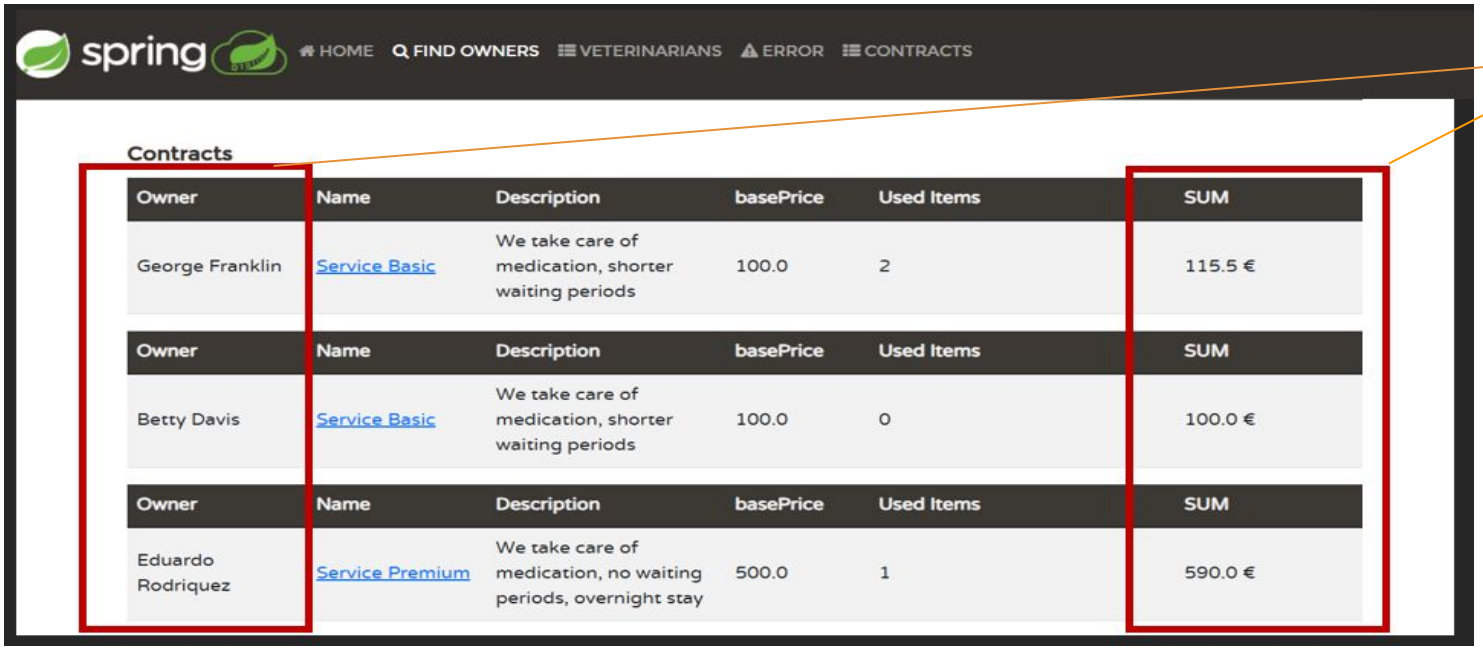
b) **Contract Details View -** Enable navigation to view all UsedItems of a selected contract

c) **Create New Contracts -** Allow adding new contracts for each owner

d) **Add Items to Contracts -** Select items from storage and convert them into UsedItems

What we did: Task 1(a)

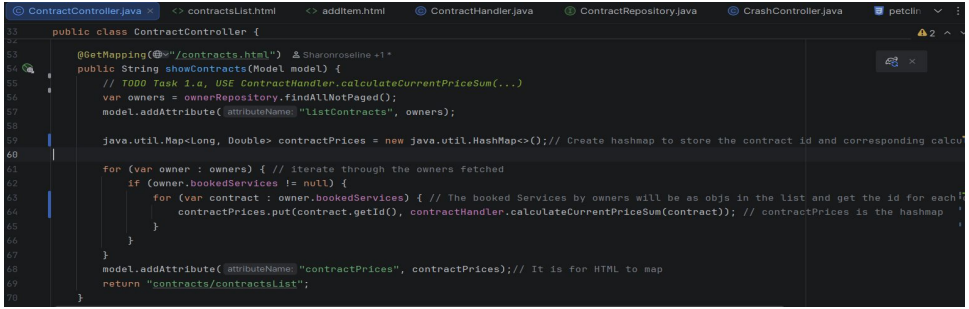
Objective: Display Contract Overview - Show contract details including owner name and total calculated price



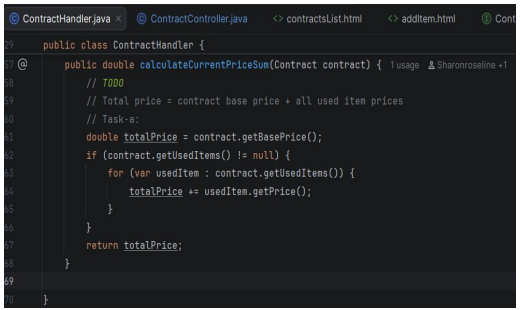
Owners + Calculated prices were added

Task 1(a): SOLUTION

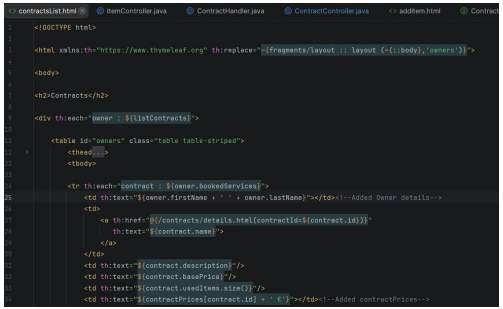
Display Contract Overview - Show contract details including owner name and total calculated price



1. ContractController.java:
- Iterated through all Owners and their Contracts
- Created a mapping (HashMap) between Contract ID and Owner
- Called ContractHandler to calculate total price for each contract
- Stored Contract ID → Total Price mapping
- Sent Owner and Price mappings to the view



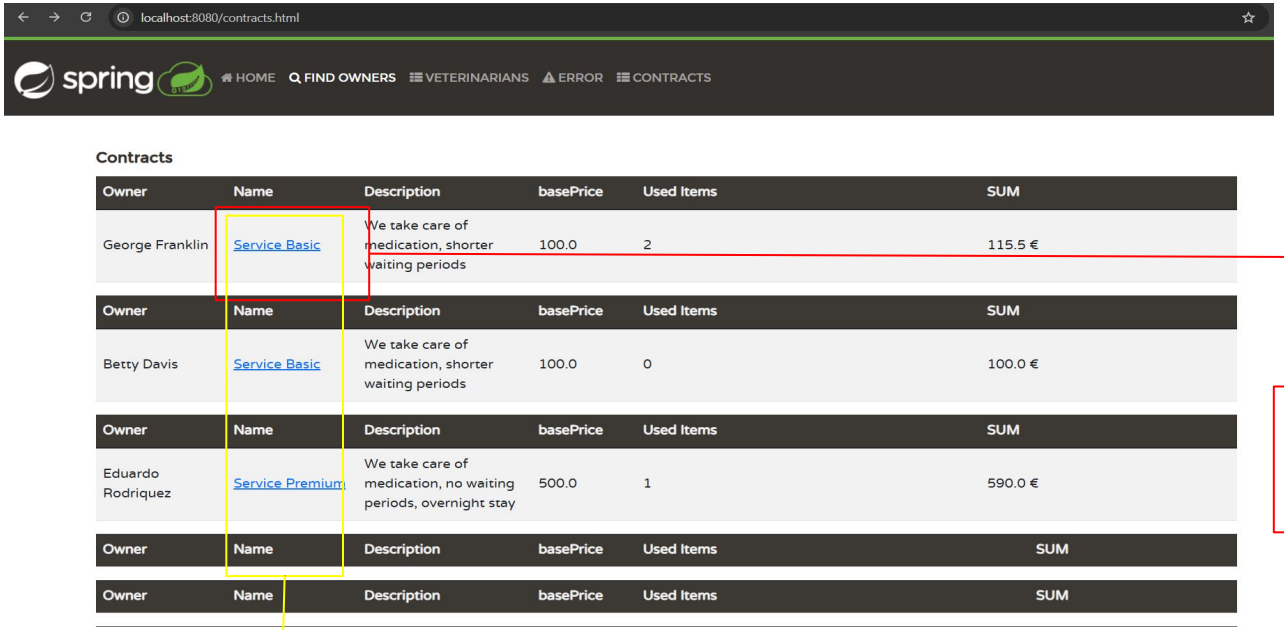
2. ContractHandler.java:
- Implemented calculateCurrentPriceSum()
- Logic:
  - Get base price of contract
  - Iterate through UsedItems
  - Sum all UsedItem prices
  - Return total price



3. contractsList.html:
- Displayed Owner details for Contracts
- Retrieved and displayed corresponding total price using Contract ID
- Used Thymeleaf to bind data from Controller

What we did: Task 1(b)

Objective:Contract Details View - Enable navigation to view all UsedItems of a selected contract





When clicking a contract, the used items and their prices are shown. (e.g. Service Basic)

Navigate to view all used items of a contract

Task 1(b): SOLUTION

Objective:Contract Details View - Enable navigation to view all UsedItems of a selected contract

Changes made:

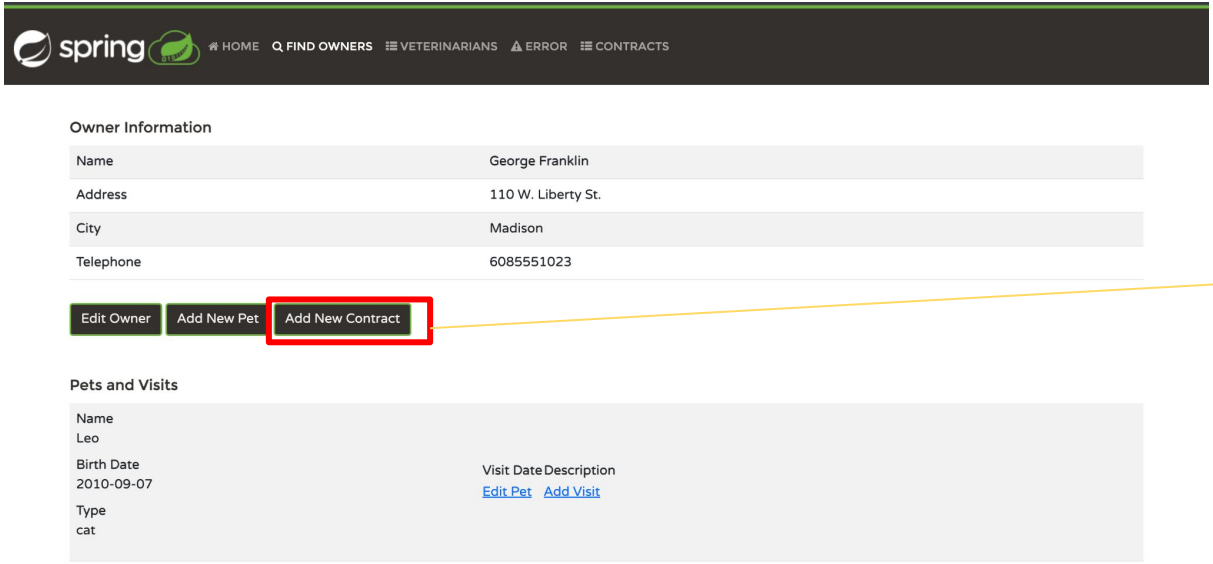
- **ContractController.java:** Added GET endpoint `/contracts/details.html` with `@RequestParam contractId` to fetch and display a specific contract
- **contractDetails.html:** Created new Thymeleaf template displaying a table of UsedItems (description & price) with a "Back to Contracts" link
- **contractsList.html:** Added clickable link on contract name navigating to `/contracts/details.html?contractId=${contract.id}`

How it works:

1. User sees the contracts list at `/contracts.html`
2. User clicks a contract name (e.g. Service Basic)
3. Controller fetches the contract by ID from the repository
4. Contract details page shows all used items and their prices

What we did: Task 1(c)

Objective: Enable Adding New Contracts to Owners - Add a form to Owner detail page to create and assign a new Contract



"Add New Contract" button added to Owner page

Task 1(c): SOLUTION

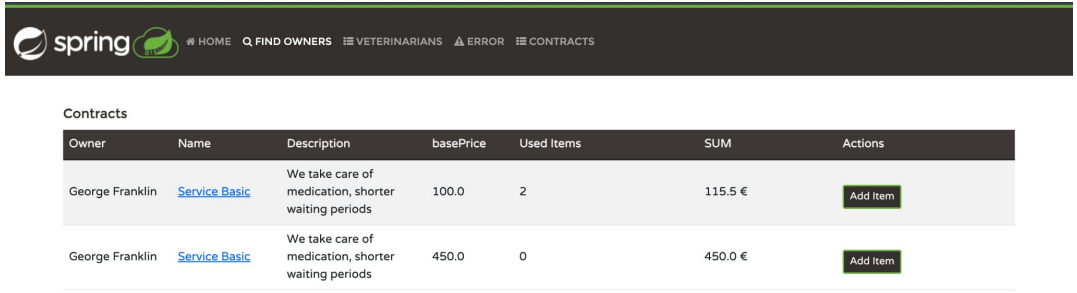
Enable Adding New Contracts to Owners - GET + POST methods in OwnerController, new addContractForm.html

1. OwnerController.java:

→ GET endpoint renders the blank contract form
→ POST receives name, description, basePrice
→ Creates new Contract, adds to owner.bookedServices
→ Saves via OwnerRepository → SQL INSERT in H2 DB

2. addContractForm.html (new file):

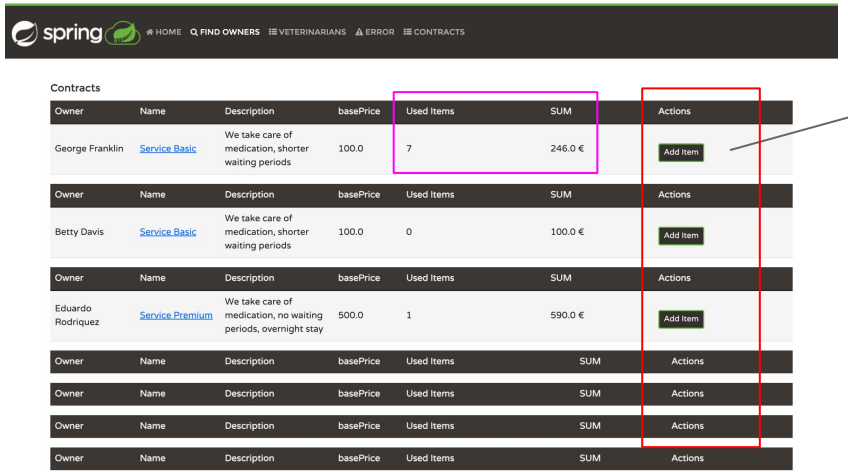
→ No existing template — created from scratch
→ Thymeleaf th:action binds to POST endpoint
→ 3 input fields: name, description, basePrice
→ Uses th:replace to match the site layout
→ Cancel button returns to owner detail page

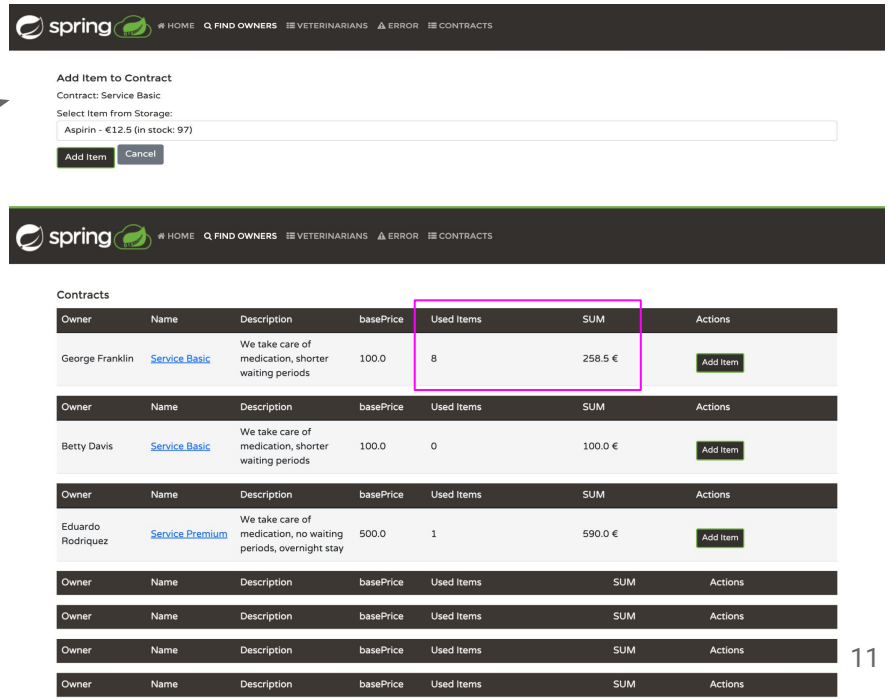


New contract appears immediately in the Contracts overview after submission

What we did: Task 1(d)

Objective: Add items from storage to contracts as UsedItems





Red box: "Add Item" button on each contract row. Clicking opens the item selection form (right).
Purple box: SUM changed from 246.0€ → 258.5€ and Used Items 7 → 8, confirming item was successfully added!

Task 1(d): SOLUTION

Objective: Add items from storage to contracts as UsedItems

1. Changes Made:

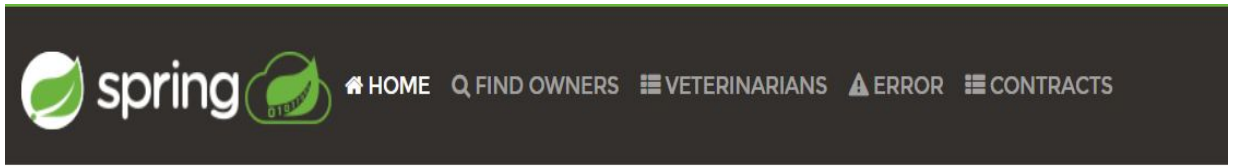
- ➔ **ItemController.java:** Added GET endpoint to show form with available storage items, and POST endpoint to handle adding selected item to contract
- ➔ **ContractHandler.java:** Implemented useItem() method to create UsedItem from Item and add to contract
- ➔ **Storage.java:** Changed from Repository<Vet, Integer> to Repository<Item, Integer>
- ➔ **contractsList.html:** Added "Add Item" button column for each contract
- ➔ **addItem.html:** Created new form page to select items from storage

2. How it works

- ➔ User clicks "Add Item" on a contract
- ➔ Form shows all available items from storage
- ➔ User selects an item and submits
- ➔ Item is saved as a UsedItem linked to the contract

Architecture

High-level View of the System



Welcome

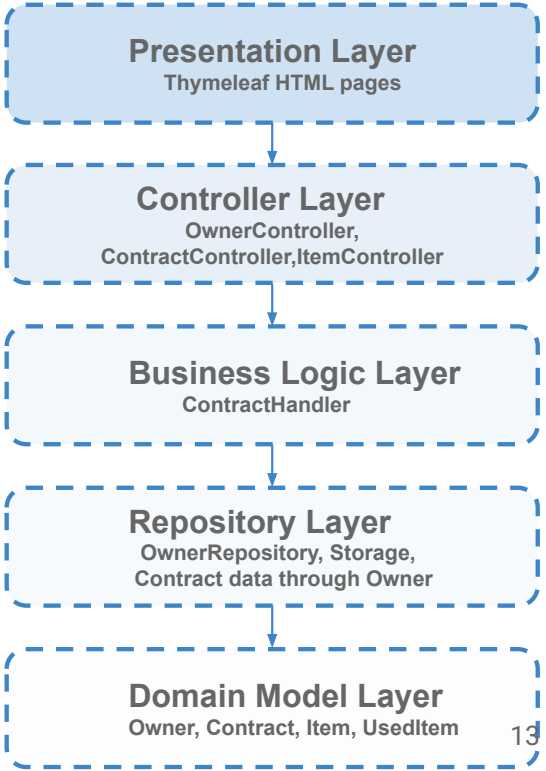


System type: Spring Boot Web application

Architecture style: Layered Monolith

Reasons:

- It is monolithic because the whole application is **deployed as one** Spring Boot application.
- It is **layered** because the user interface, controllers, business logic, repository logic, and domain models are **separated**.



DESIGN

Low-level View of the System

Design defines **“how the system is implemented internally”**

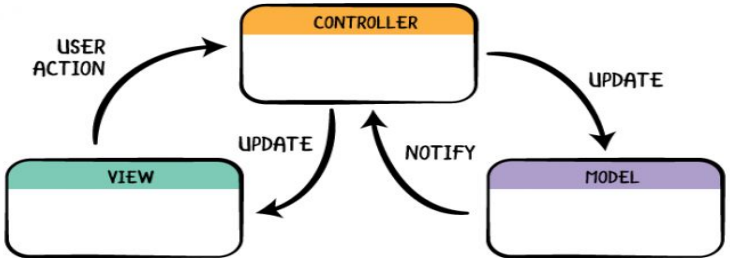
Design style: Model-View-Controller(MVC) and **Service/Handler Pattern**

Reason: The system mainly follows the MVC pattern where:

- **Models** represent domain entities
- **Views** display data
- **Controllers** handle requests

Also follows Service/Handler Pattern where:

- Handles business logic
- Keeps controllers clean
- Centralizes logic

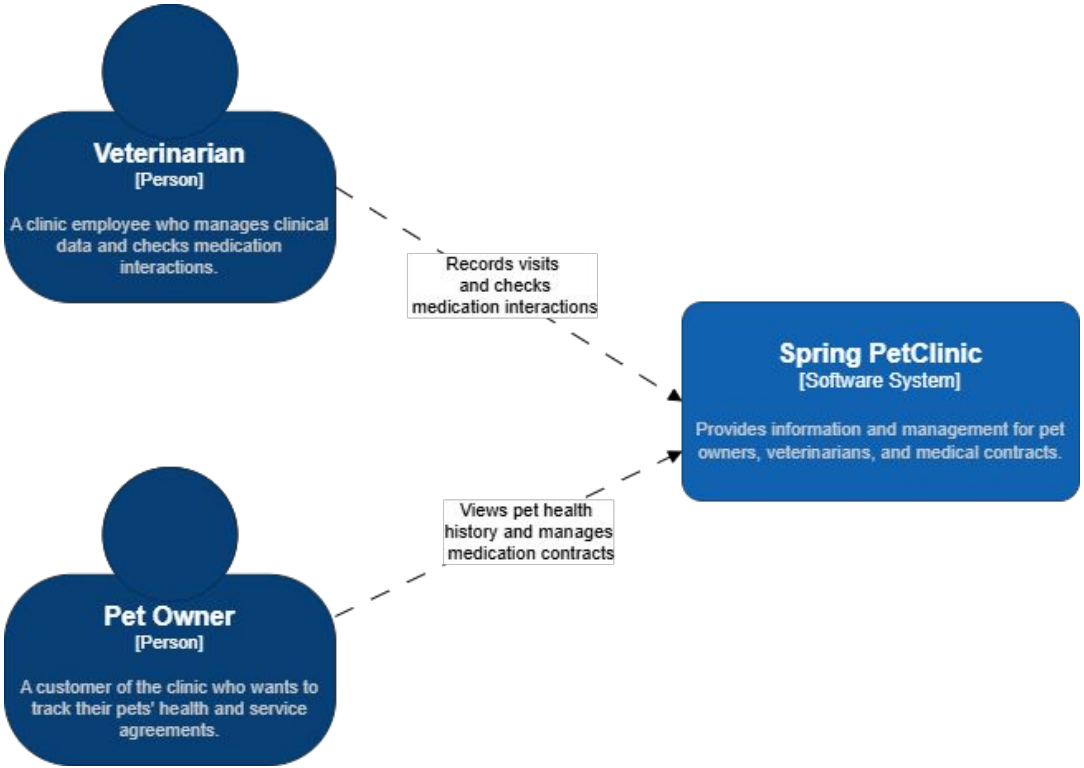


Design level also focuses on classes, methods, relationships, and responsibilities.

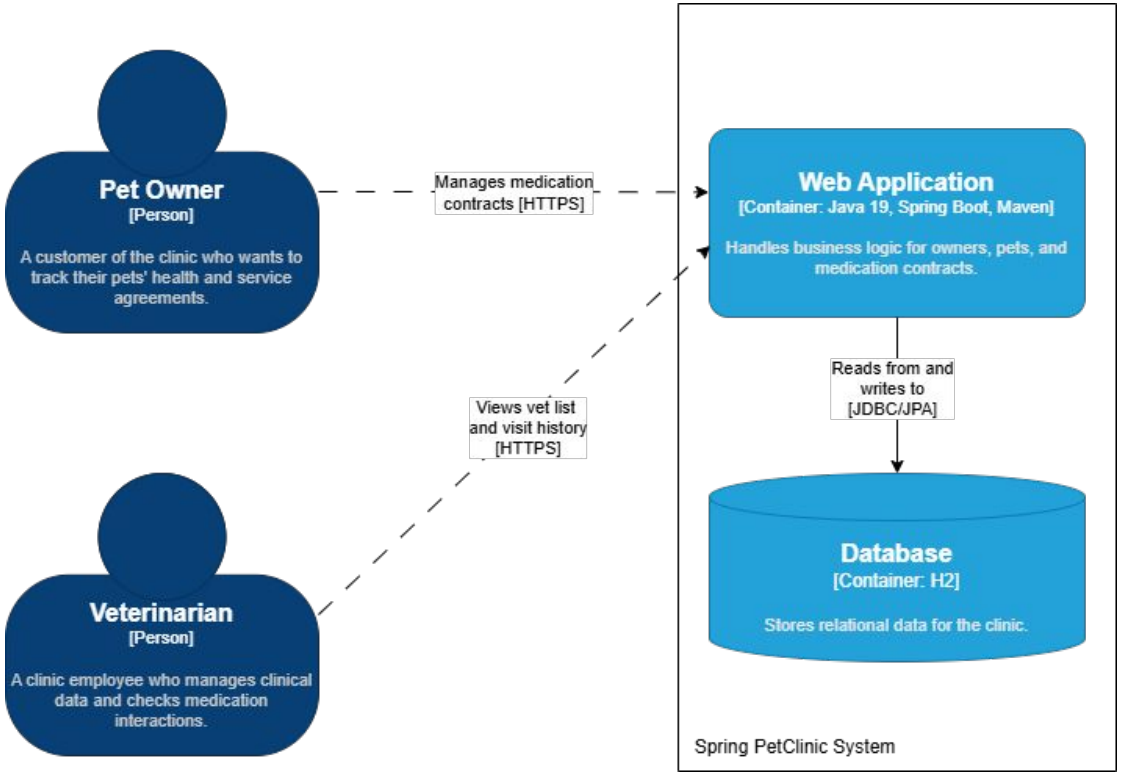
- ★ **Owner → many Contracts**
- ★ **Contract → many UsedItems**
- ★ **Storage → many Items**
- ★ **Item → converted into UsedItem when added to Contract**
- ★ ContractHandler performs price calculation and item usage logic
- ★ Controllers handle user actions and page navigation

https://www.kodeco.com/1000705-model-view-controller-mvc-in-ios-a-modern-approach

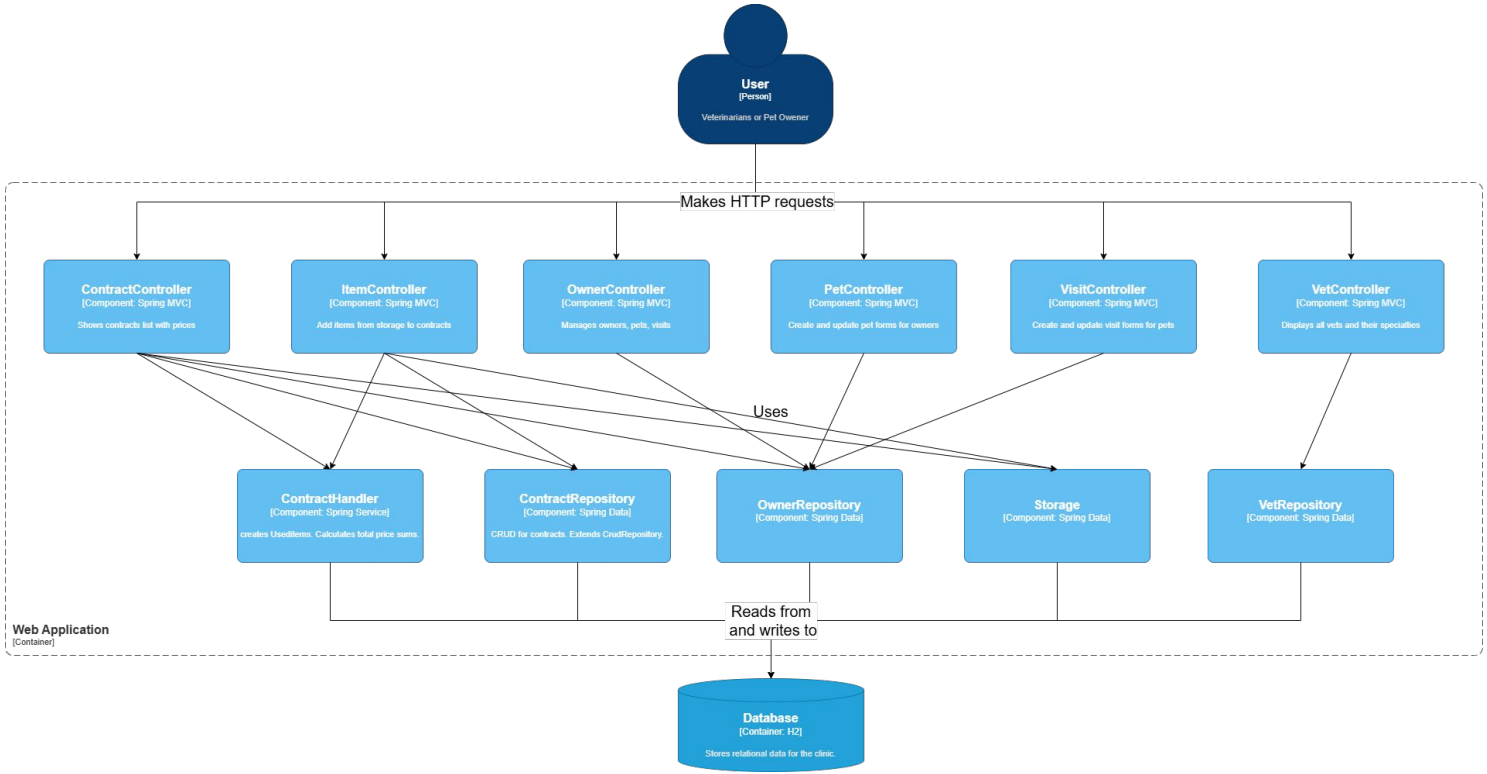
System Diagram



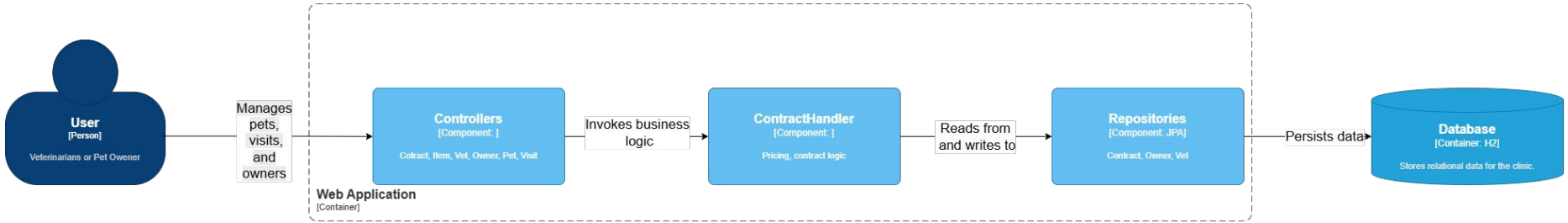
Container Diagram



Component Diagram



Dynamic Diagram



Code Quality

Existing Aspects:

- Layered approach - improved separation of concerns
- Existing PetClinic code is readable and follows common Spring conventions
- ContractHandler separates some business logic from controllers

Limitations:

- Some business logic is present inside controllers
- ContractHandler.startContract() is incomplete / not fully used
- Error handling is limited for invalid owner ID, contract ID, or item ID
- Validation is missing for contract forms and item stock availability
- Some data access is indirect, for example contracts are accessed through owners
- New feature tests are not fully updated

Code Quality

Scalability / Maintainability:

- The existing system is scalable only to a limited extent.
- It is suitable for a small clinic-style application because it follows a layered Spring Boot structure and separates controllers, views, repositories, and domain models.
- It is not highly scalable for large systems because it is still a monolith. As more features are added, the controllers and domain relationships can become complex. The contract feature also shows this, because contract data is handled through owners and some logic remains inside controllers.
- To improve scalability and to become future-proof, the system should introduce clearer service layers, dedicated repositories, better validation, and stronger modularization.

Testing Analysis

- **What is tested?**
- The existing test suite mainly tests the original PetClinic features:
  - Owner, Pet, Visit, and Vet workflows
  - Validation logic
  - Service/repository behavior
  - Basic application startup

- **What is really tested?**
- Mostly the standard PetClinic functionality is tested. The newly added contract feature is only partially covered because the existing tests were written before our changes.

- **How is it tested?**
- **Unit Testing:**
  • Individual logic such as validation and service methods
- **Integration Testing:**
  • Spring context, repositories, controllers, and application startup
- **System Testing:**
  • Basic user workflows can be checked manually through the UI

Testing Setup and Realism

Necessary setup

- Java 19 or above
- IntelliJ IDEA
- Spring Boot test framework
- JUnit / MockMvc test structure
- H2 in-memory database for test execution

Are test cases realistic?

Yes, for the original PetClinic system.

They represent realistic workflows such as:

- Viewing owners
- Managing pets
- Adding visits
- Viewing veterinarians

For our contract feature, realistic test cases should include:

- View contract overview
- Check owner name and calculated price
- Open contract details
- Create a new contract for an owner
- Add storage item to contract

Coverage and Gaps

Coverage Through Manual analysis

Line coverage:

- Existing tests execute many original controller, service, validation, and repository paths.
- New contract-related code paths are not fully covered.

Branch coverage:

- Standard success paths are mostly tested.
- Error and edge-case branches are limited.

Important branches that should be checked:

- Contract has UsedItems / no UsedItems
- Valid contract ID / invalid contract ID
- Valid owner ID / invalid owner ID
- Item exists / item missing
- Item in stock / item out of stock

What is not tested?

- Contract overview owner-price mapping
- Total price calculation with zero, one, and multiple UsedItems
- Contract details page with UsedItems
- New contract creation for owner
- Item to UsedItem conversion
- Invalid IDs and missing data cases
- Out-of-stock item behavior

Why Not Tested and Should It Be Tested?

Why is it not tested?

- The original tests were created for the standard PetClinic system.
- Our contract feature was added later, so the existing test suite has not yet been updated to fully cover the new behavior.

Should it be tested? Yes.

- The contract feature should be tested because it contains important business logic:
  a. Price calculation
  b. Owner-contract relationship
  c. Contract details display
  d. Item to UsedItem conversion

Final conclusion

- The original PetClinic system has a useful test base, but the new contract-management feature requires additional focused tests to improve robustness and maintainability.